

Software engineering

Architectures

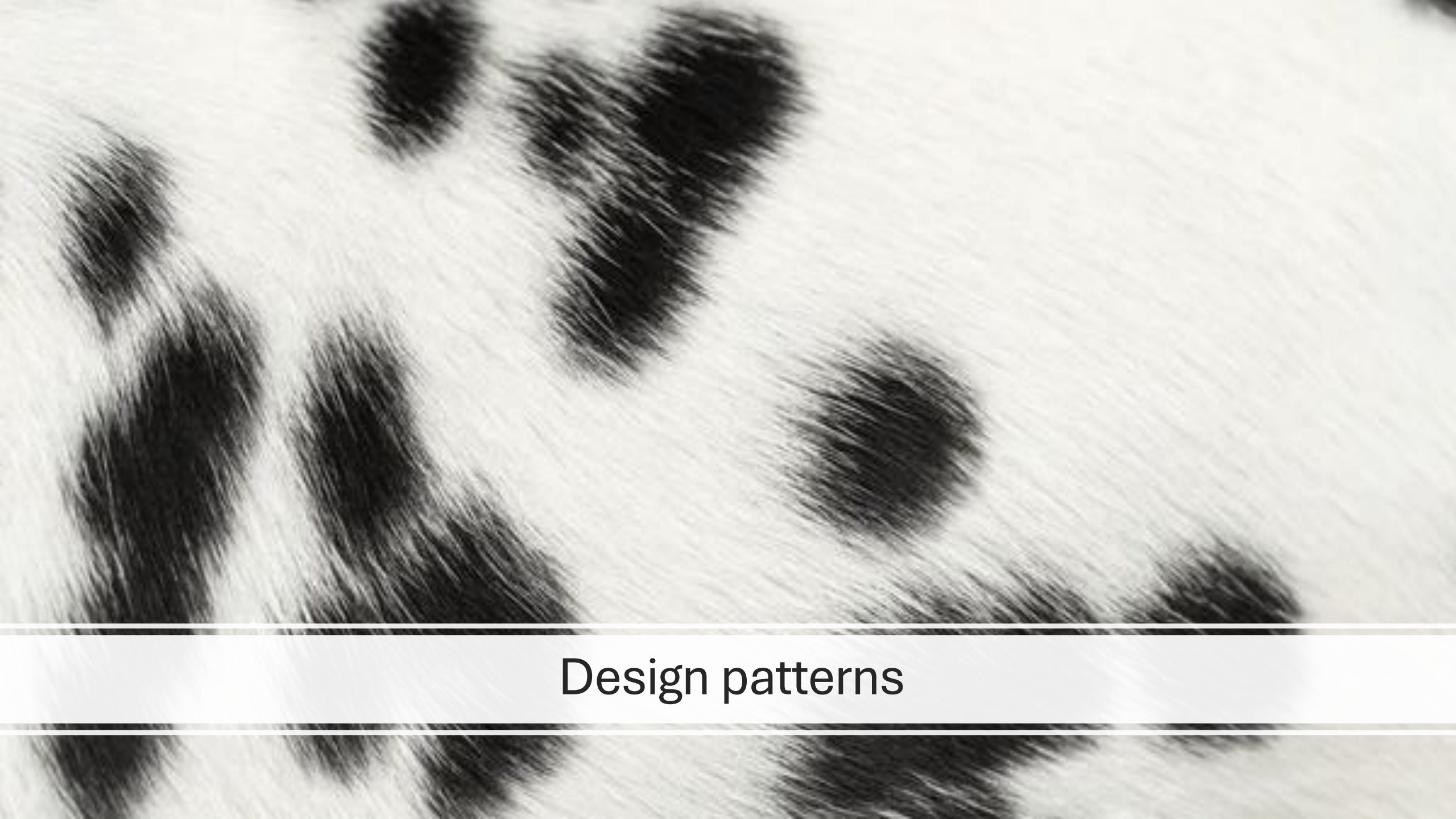
Skill progressions by applying material:

Software architectures +3

Design patterns +2-3

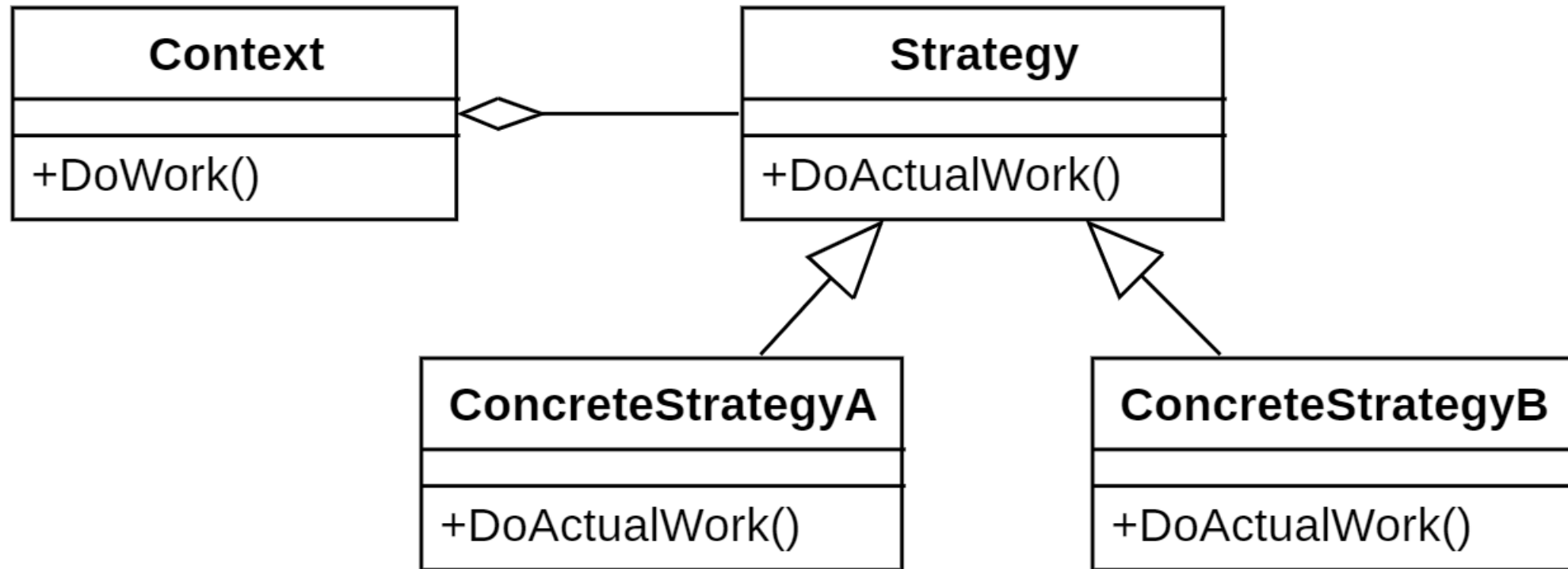
About the last 2 weeks

- 3 seminars fall after the exam, so we'll just cancel the other 4 😊
- Last year students finish their semester this week
- Only the second lab will be held for each team
 - 921/1 + 923/1 => Week 13, Tuesday 12:00 L343
 - 922/1 + 927/1 => Week 13, Tuesday 18:00 L439
 - 924/1 + 924/2 => Week 13, Thursday 18:00 MOS-S15
 - 926/2 + 921/2 => Week 14, Monday 12:00 MOS-S15
 - 923/2 + 922/2 => Week 14, Tuesday 14:00 L339
 - 927/2 + 925/2 => Week 14, Thursday 16:00 MOS-S15
 - 926/1 + 925/1 => Week 14, Friday 18:00 L307



Design patterns

Strategy



```

public abstract class Strategy
{
    3 references
    public abstract void DoActualWork();
}

1 reference
public class ConcreteStrategyA : Strategy
{
    2 references
    public override void DoActualWork()
    {
        Console.WriteLine("Doing work A");
    }
}

1 reference
public class ConcreteStrategyB : Strategy
{
    2 references
    public override void DoActualWork()
    {
        Console.WriteLine("Doing work B");
    }
}

```

```

public class Context
{
    Strategy strategy;

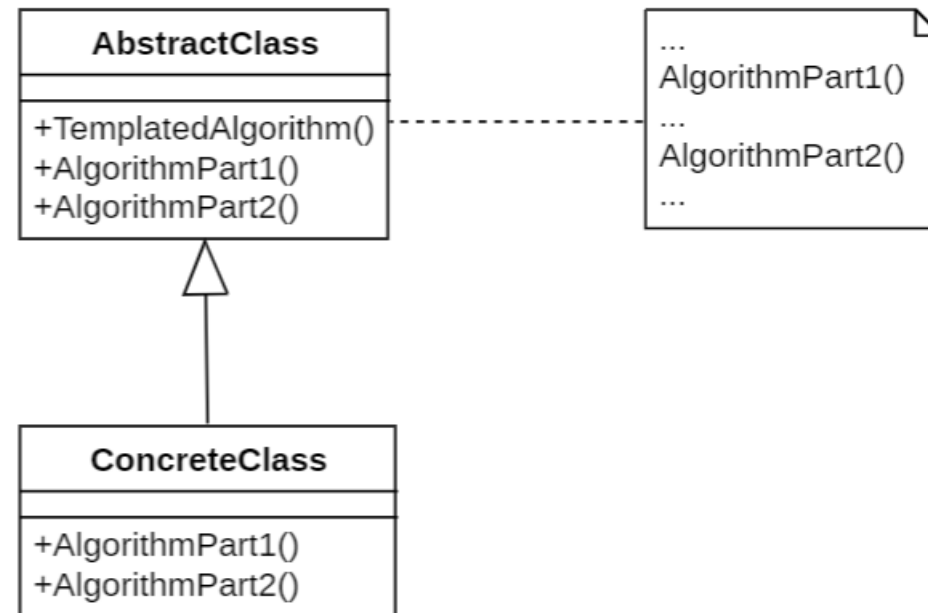
    2 references
    public Context(Strategy strategy) //usually injected
    {
        this.strategy = strategy;
    }

    2 references
    public void DoWork()
    {
        strategy.DoActualWork();
    }

    0 references
    public static void Main(string[] args)
    {
        new Context(new ConcreteStrategyA()).DoWork();
        new Context(new ConcreteStrategyB()).DoWork();
    }
}

```

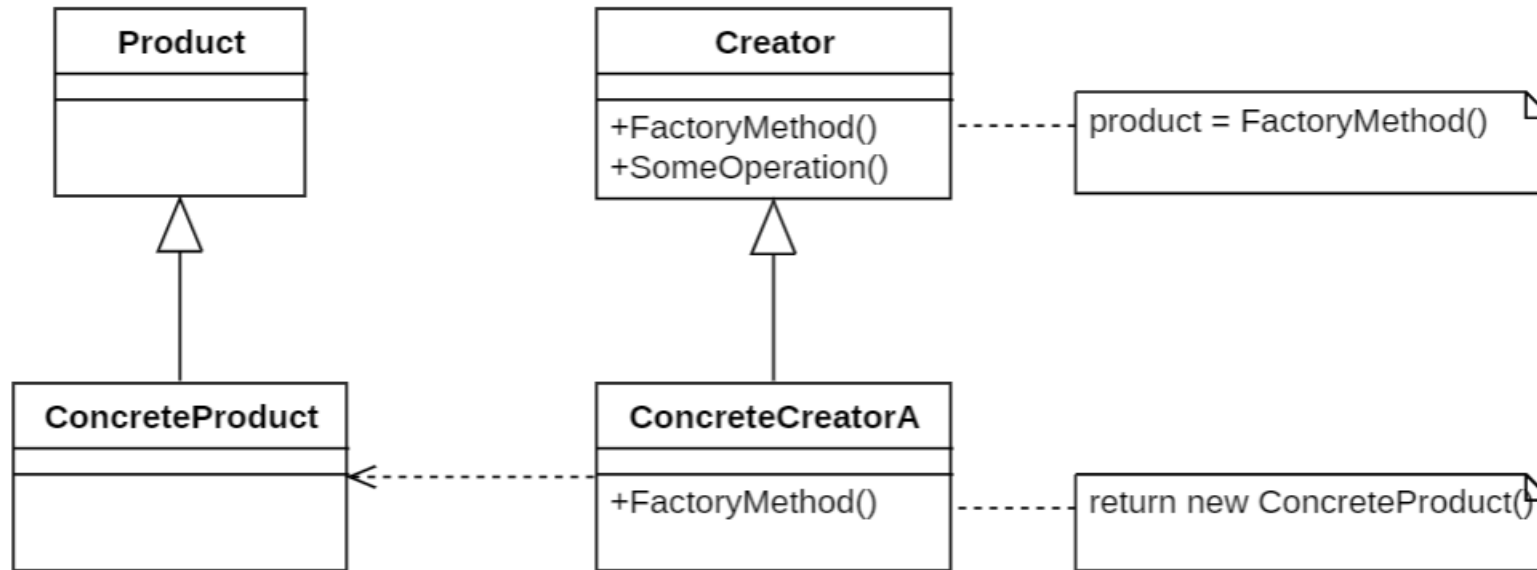
Template



```
public abstract class AbstractClass
{
    2 references
    public abstract void AlgorithmPart1();
    2 references
    public abstract void AlgorithmPart2();
    0 references
    public void TemplatedAlgorithm()
    {
        AlgorithmPart1();
        AlgorithmPart2();
    }
}
```

```
public class ConcreteClass : AbstractClass
{
    2 references
    public override void AlgorithmPart1()
    {
        Console.WriteLine("Do a part");
    }
    2 references
    public override void AlgorithmPart2()
    {
        Console.WriteLine("Do another part");
    }
}
```

Factory Method



```

abstract class Product { }

class ConcreteProductA : Product { }

class ConcreteProductB : Product { }

abstract class Creator
{
    public abstract Product FactoryMethod();
}

class ConcreteCreatorA : Creator
{
    public override Product FactoryMethod()
    {
        return new ConcreteProductA();
    }
}

class ConcreteCreatorB : Creator
{
    public override Product FactoryMethod()
    {
        return new ConcreteProductB();
    }
}

```

```

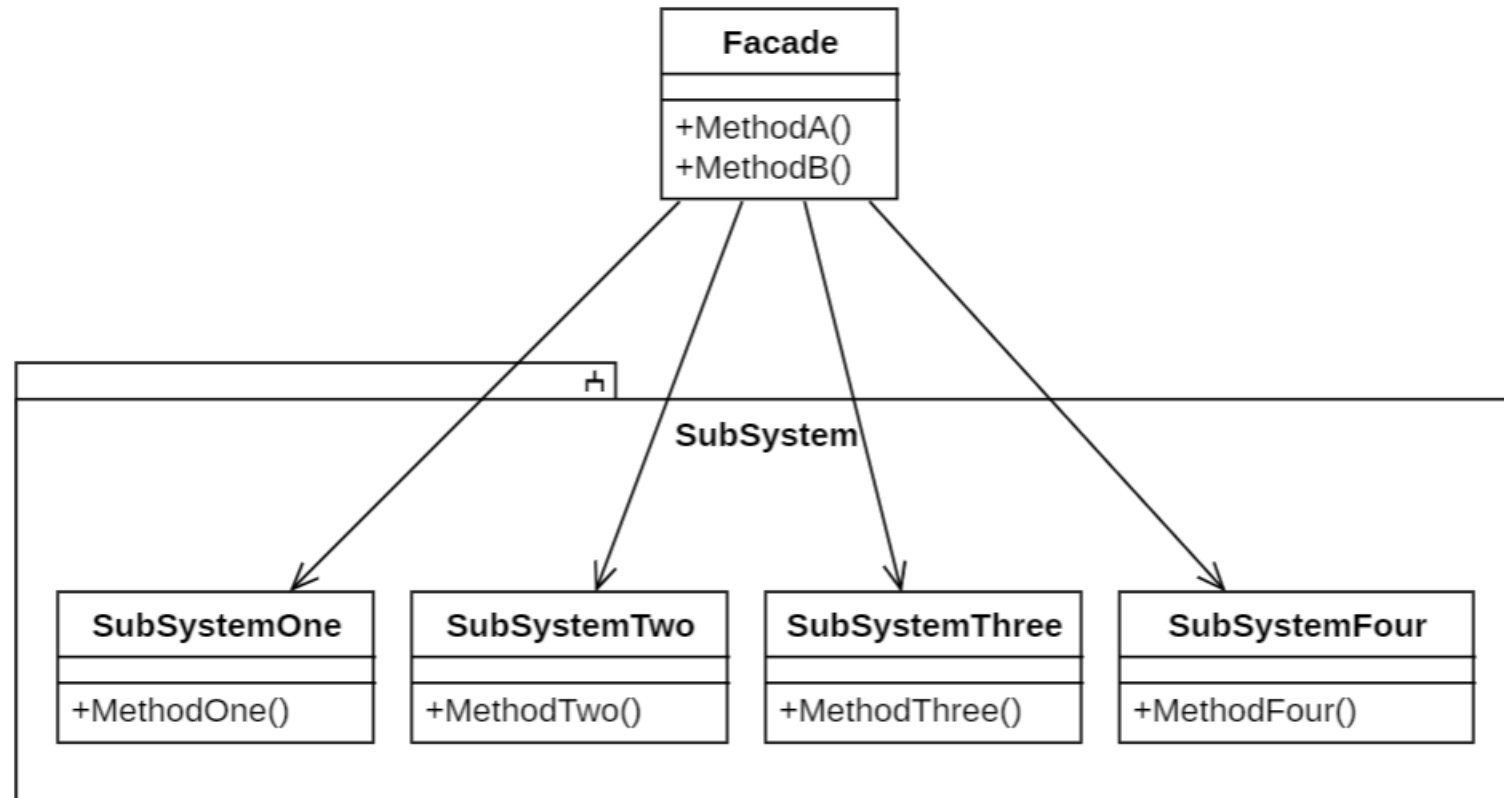
class FactoryMethodExample
{
    static void Main()
    {
        Creator[] creators = new Creator[2];

        creators[0] = new ConcreteCreatorA();
        creators[1] = new ConcreteCreatorB();

        foreach (Creator creator in creators)
        {
            Product product = creator.FactoryMethod();
            Console.WriteLine("Created {0}",
                product.GetType().Name);
        }
    }
}

```

Facade



```

public class Facade
{
    SubSystemOne one;
    SubSystemTwo two;
    SubSystemThree three;
    SubSystemFour four;

    public Facade()
    {
        one = new SubSystemOne();
        two = new SubSystemTwo();
        three = new SubSystemThree();
        four = new SubSystemFour();
    }

    public void MethodA()
    {
        one.MethodOne();
        two.MethodTwo();
        four.MethodFour();
    }

    public void MethodB()
    {
        two.MethodTwo();
        three.MethodThree();
    }
}

```

```

Facade facade = new Facade();

facade.MethodA();
facade.MethodB();
}

public class SubSystemOne
{
    public void MethodOne() { }
}

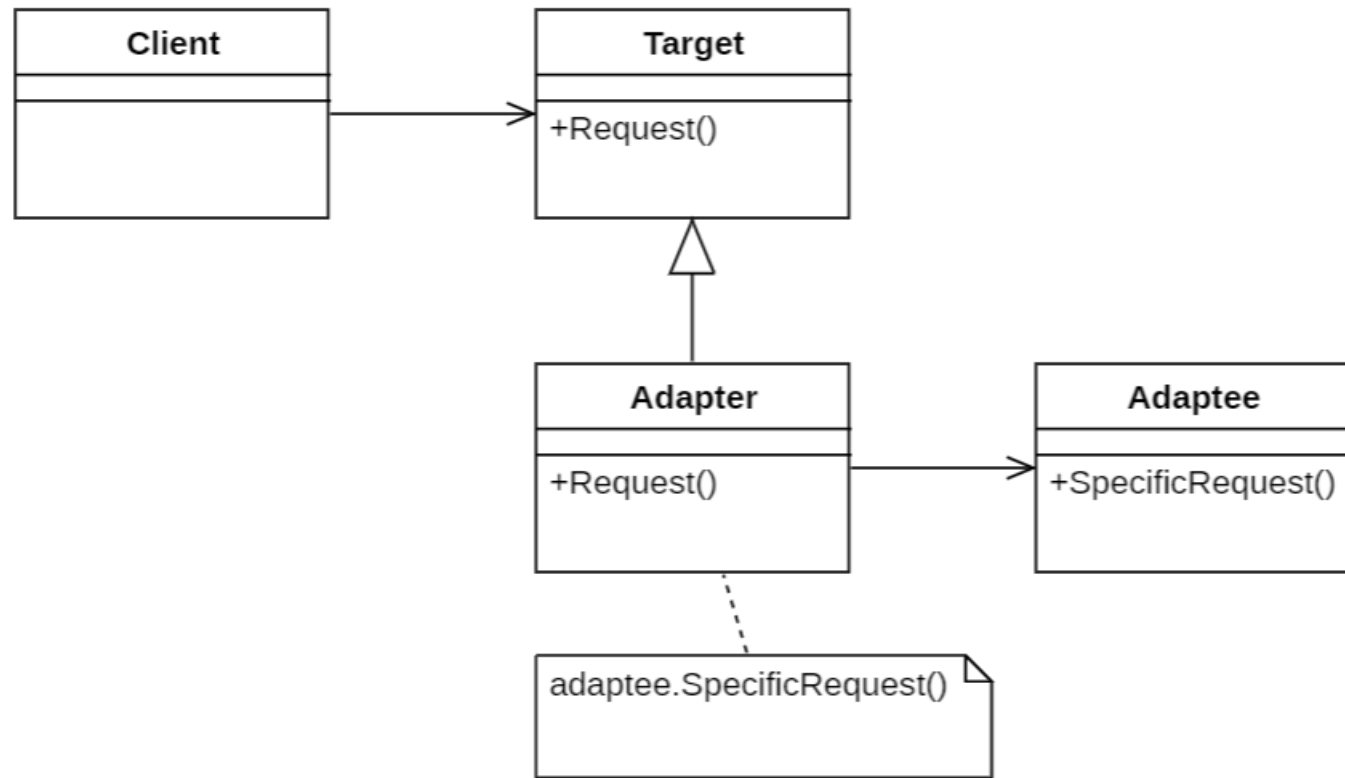
public class SubSystemTwo
{
    public void MethodTwo() { }
}

public class SubSystemThree
{
    public void MethodThree() { }
}

public class SubSystemFour
{
    public void MethodFour() { }
}

```

Adapter



```
public class Target
{
    public virtual void Request() { }
}

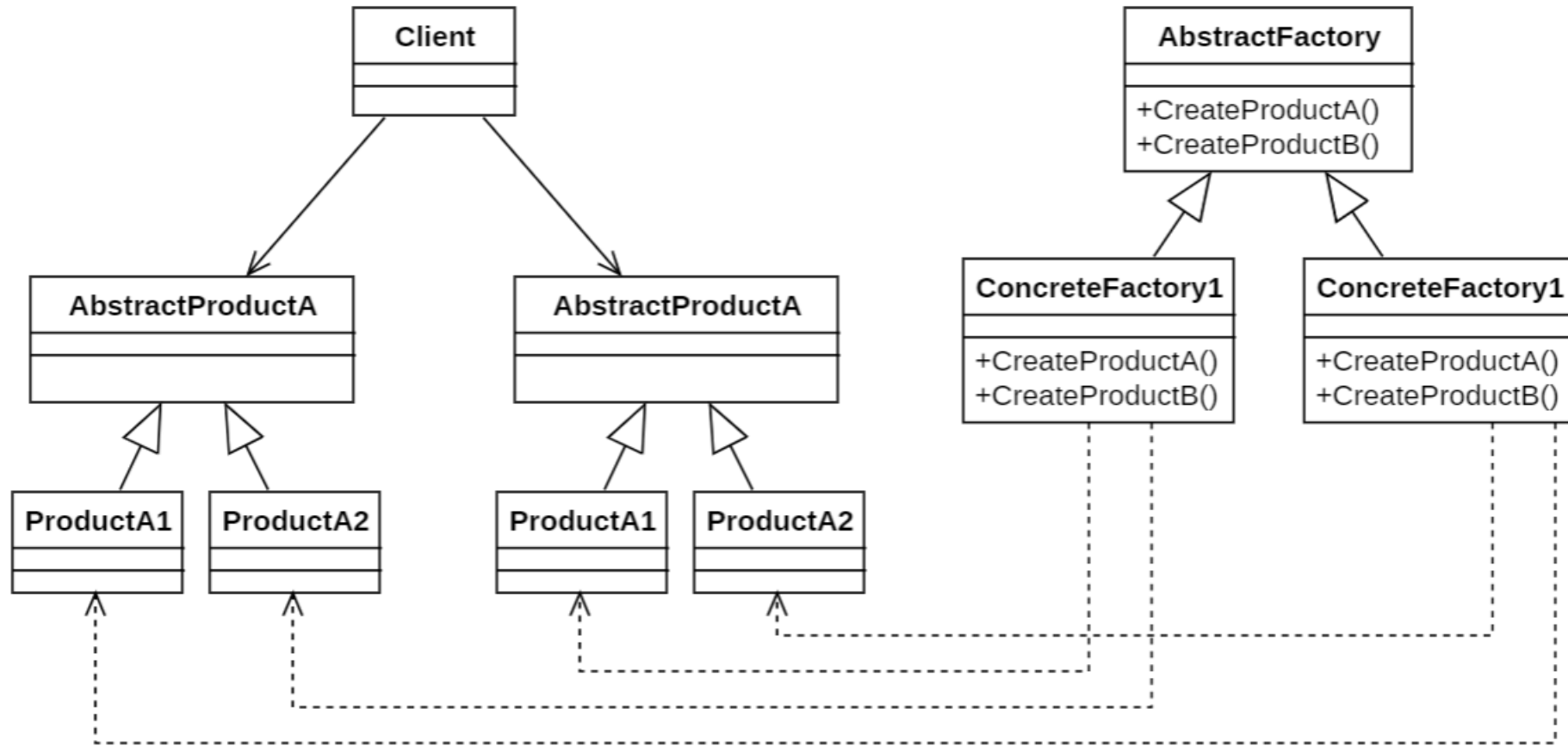
public class Adaptee
{
    public void SpecificRequest() { }
}

public class Adapter : Target
{
    private Adaptee adaptee = new Adaptee();

    public override void Request()
    {
        // Possibly do other work
        adaptee.SpecificRequest();
    }

    public static void Main(string[] args)
    {
        Target target = new Adapter();
        target.Request();
    }
}
```

Abstract factory



```

abstract class AbstractFactory
{
    public abstract AbstractProductA CreateProductA();
    public abstract AbstractProductB CreateProductB();
}

class ConcreteFactory1 : AbstractFactory
{
    public override AbstractProductA CreateProductA()
    {
        return new ProductA1();
    }
    public override AbstractProductB CreateProductB()
    {
        return new ProductB1();
    }
}

class ConcreteFactory2 : AbstractFactory
{
    public override AbstractProductA CreateProductA()
    {
        return new ProductA2();
    }
    public override AbstractProductB CreateProductB()
    {
        return new ProductB2();
    }
}

abstract class AbstractProductA { }

class ProductA1 : AbstractProductA { }

class ProductB1 : AbstractProductB { }

```

```

abstract class AbstractProductB { }

class ProductA2 : AbstractProductA { }

class ProductB2 : AbstractProductB { }

class Client
{
    private AbstractProductA _abstractProductA;
    private AbstractProductB _abstractProductB;

    public Client(AbstractFactory factory)
    {
        _abstractProductB = factory.CreateProductB();
        _abstractProductA = factory.CreateProductA();
    }

    public void Run()
    {
        //_abstractProductB.Interact(_abstractProductA);
    }

    public static void Main()
    {
        AbstractFactory factory1 = new ConcreteFactory1();
        Client client1 = new Client(factory1);
        client1.Run();

        AbstractFactory factory2 = new ConcreteFactory2();
        Client client2 = new Client(factory2);
        client2.Run();
    }
}

```



Clean architecture



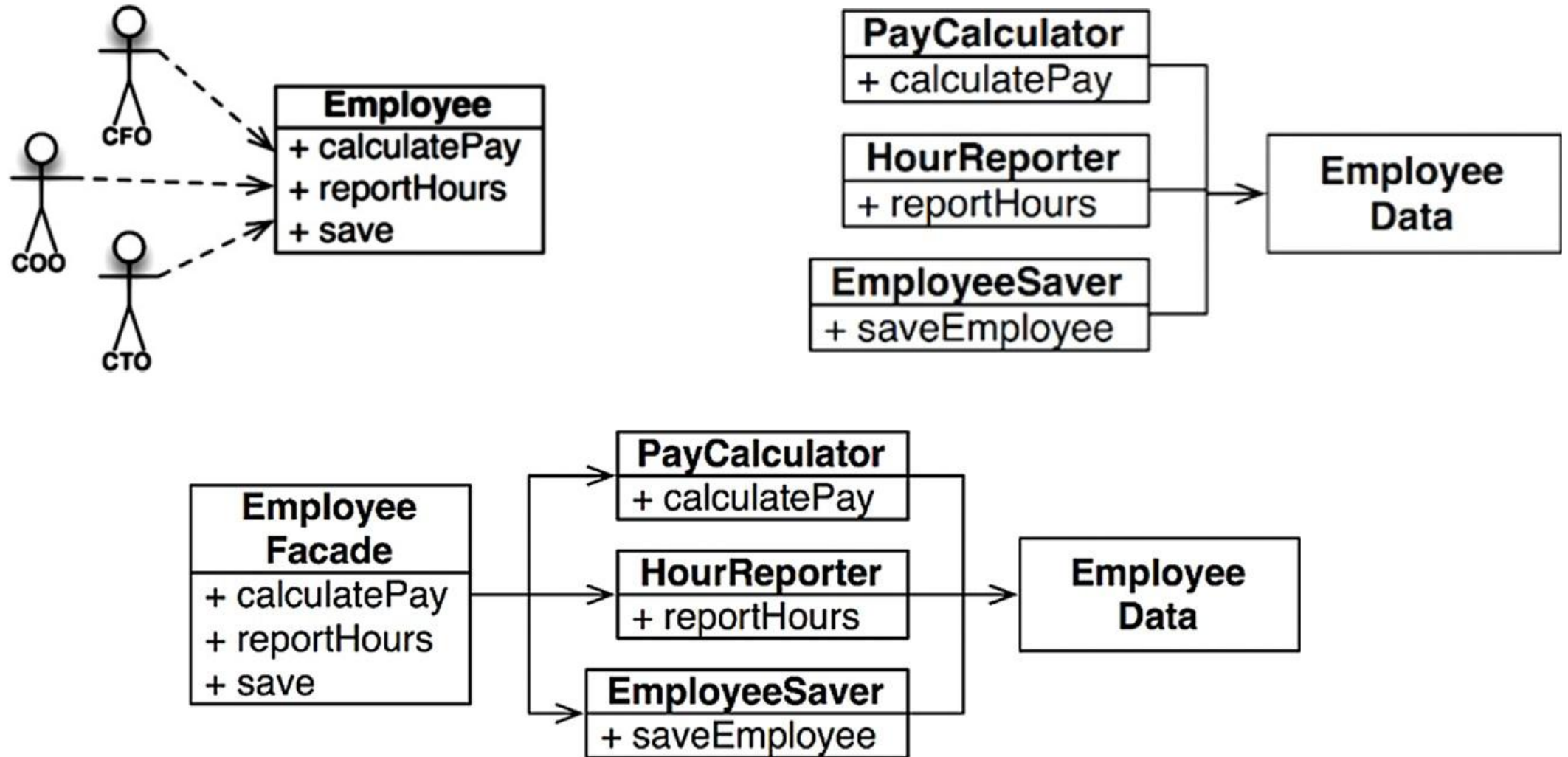


S.O.L.I.D.

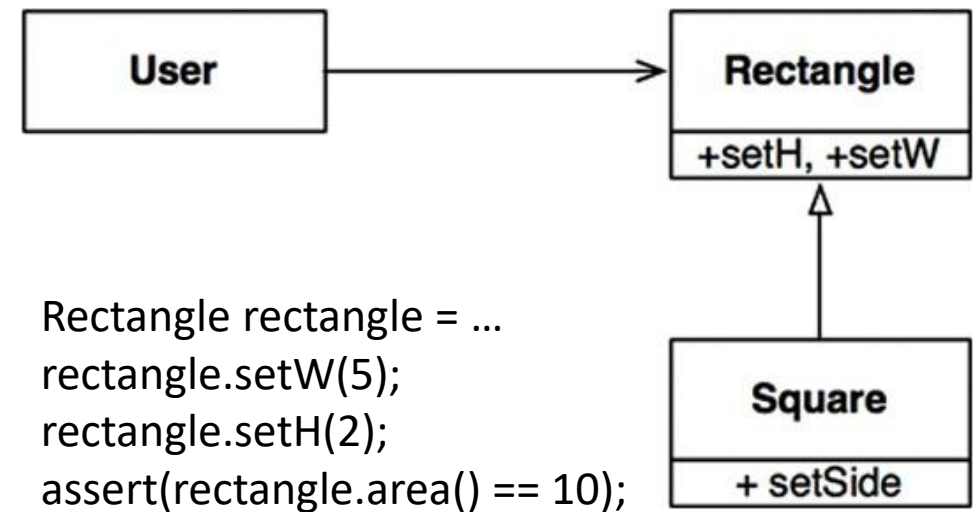
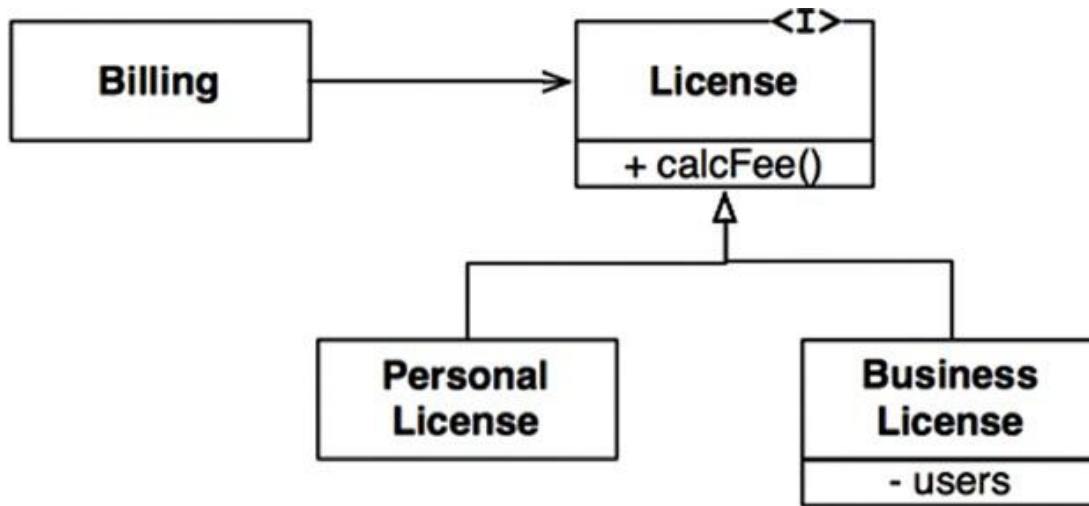


- **S**ingle Responsibility Principle
 - Each software module has one, and only one, reason to change.
- **O**pen-Closed Principle
 - It's better to add new code than change existing one.
- **L**iskov Substitution Principle
 - Build software using interchangeable contracts.
- **I**nterface Segregation Principle
 - Avoid depending on things that you don't use.
- **D**ependency Inversion Principle
 - High-level policy should not depend on the code that implements low-level details.

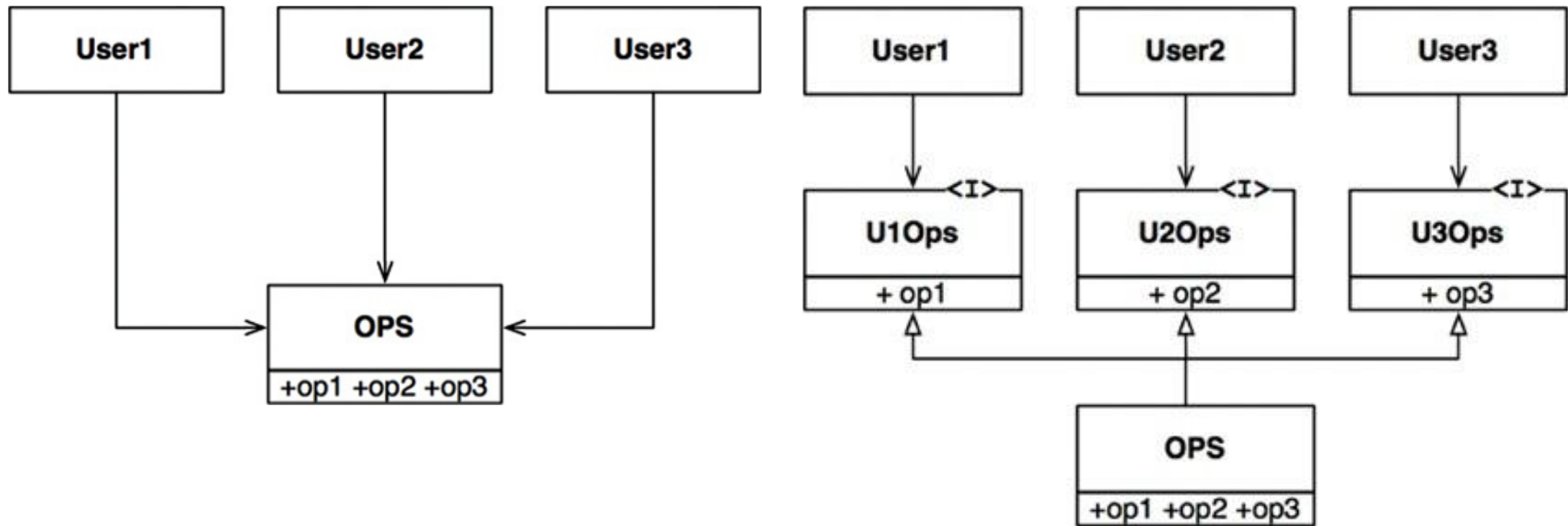
Single Responsibility Principle: Each software module has one, and only one, reason to change.



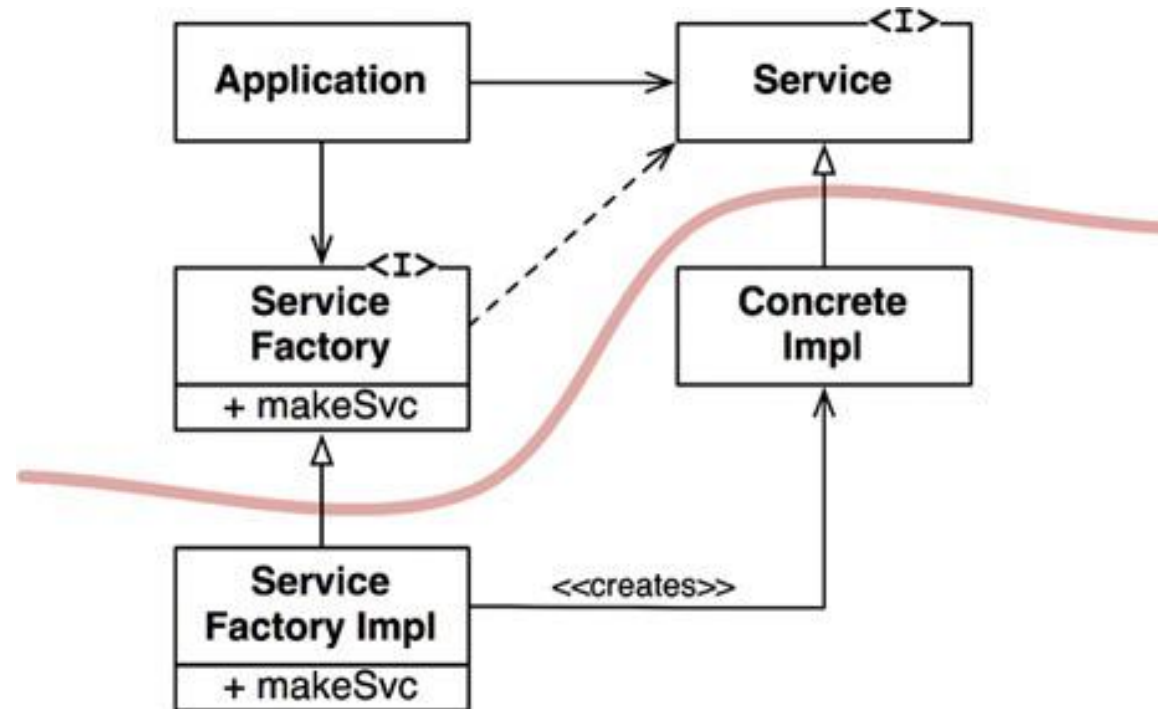
Liskov Substitution Principle: Build software using interchangeable contracts.



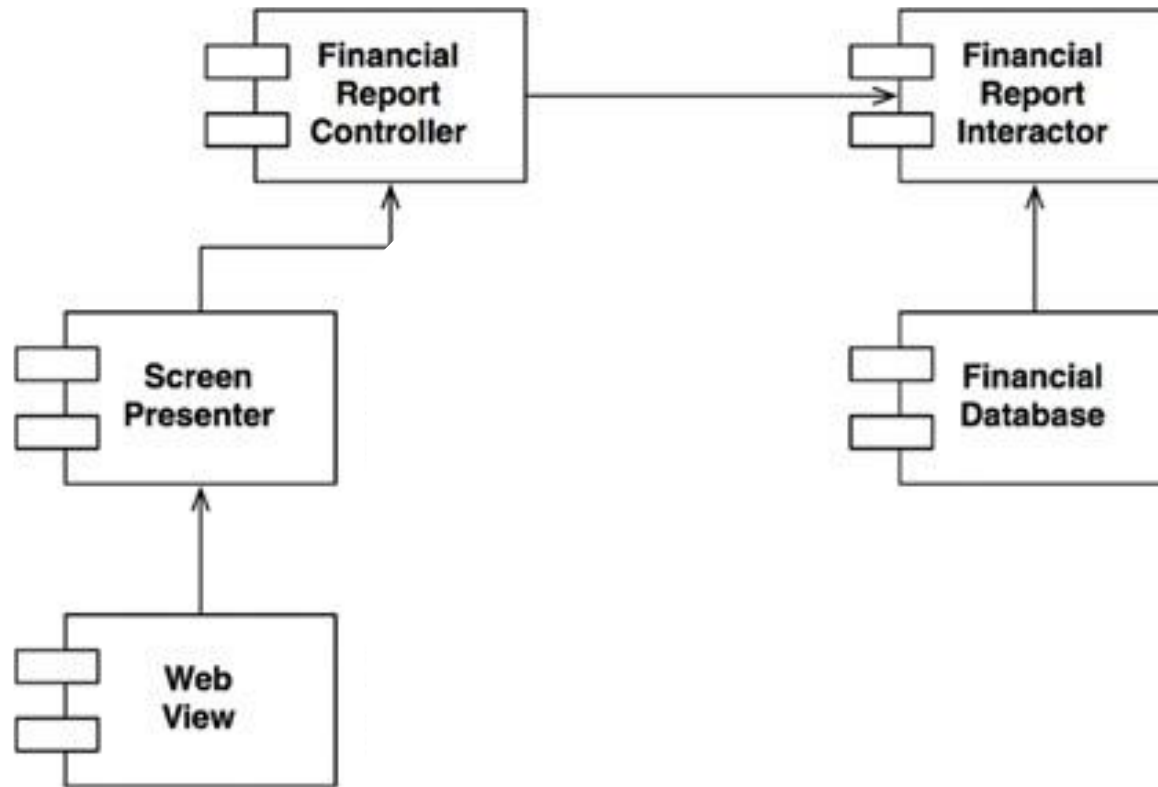
Interface Segregation Principle: Avoid depending on things that you don't use.

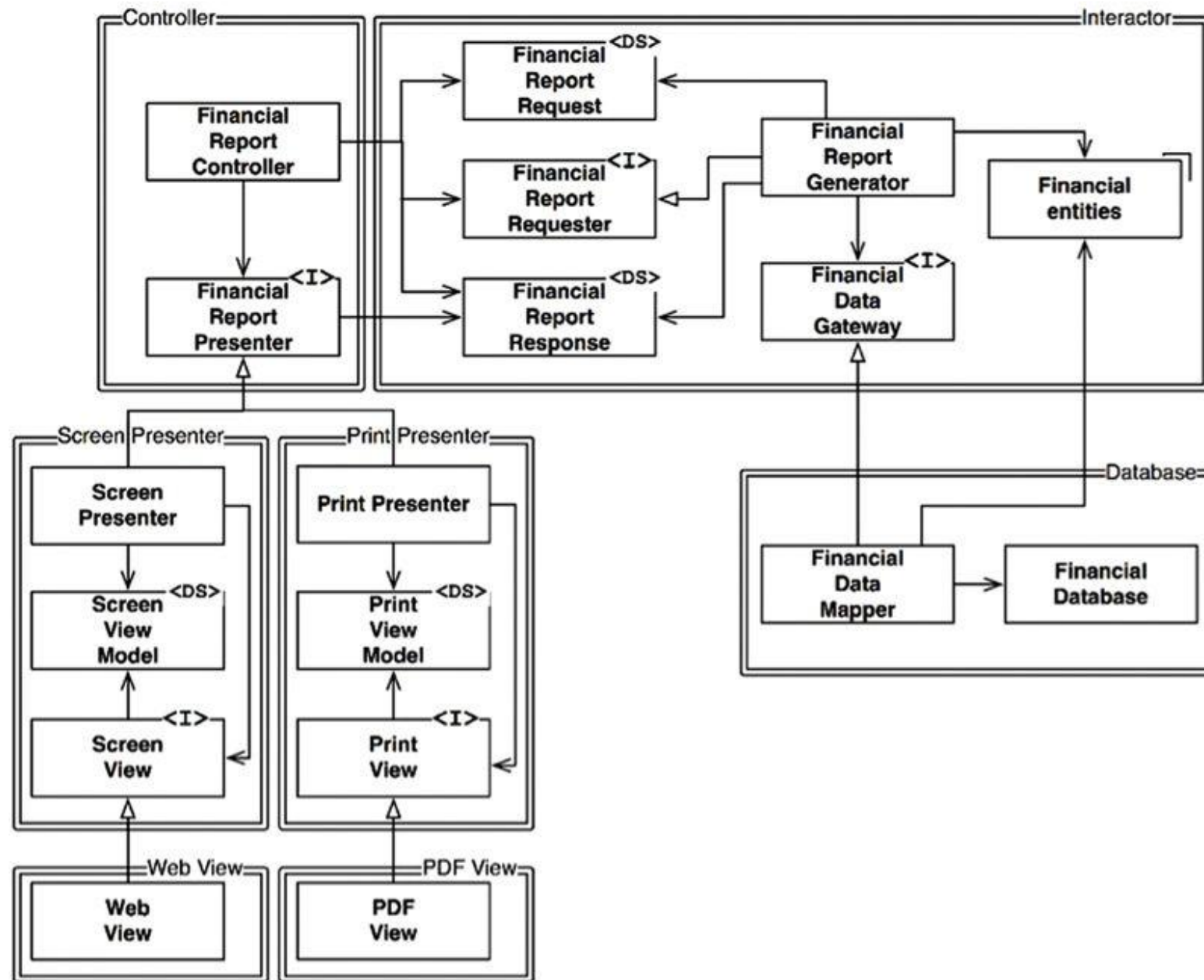


Dependency Inversion Principle: High-level policy should not depend on the code that implements low-level details.



Open-Closed Principle: It's better to add new code than change existing one.

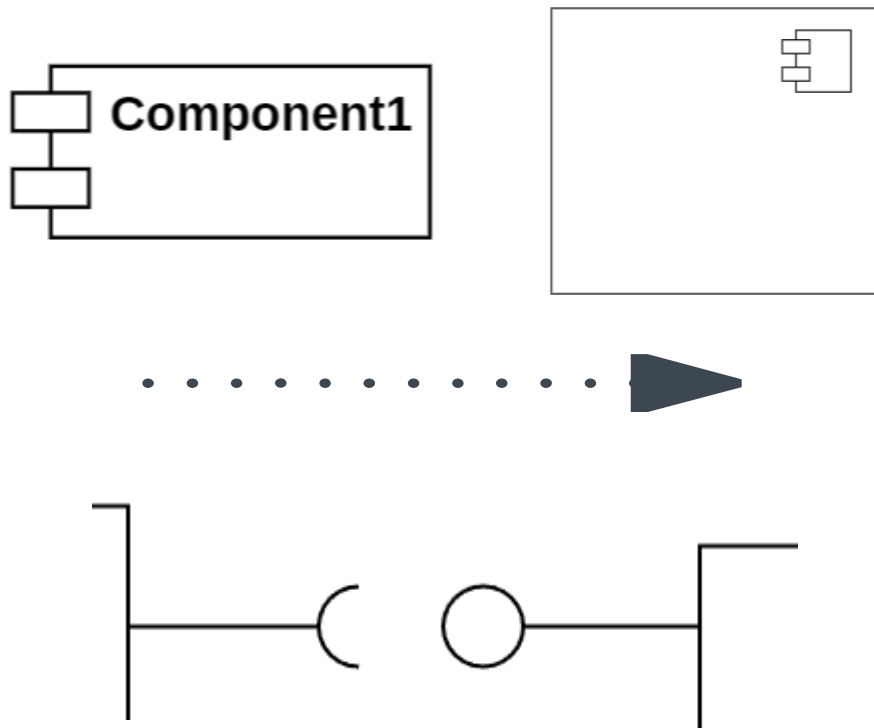




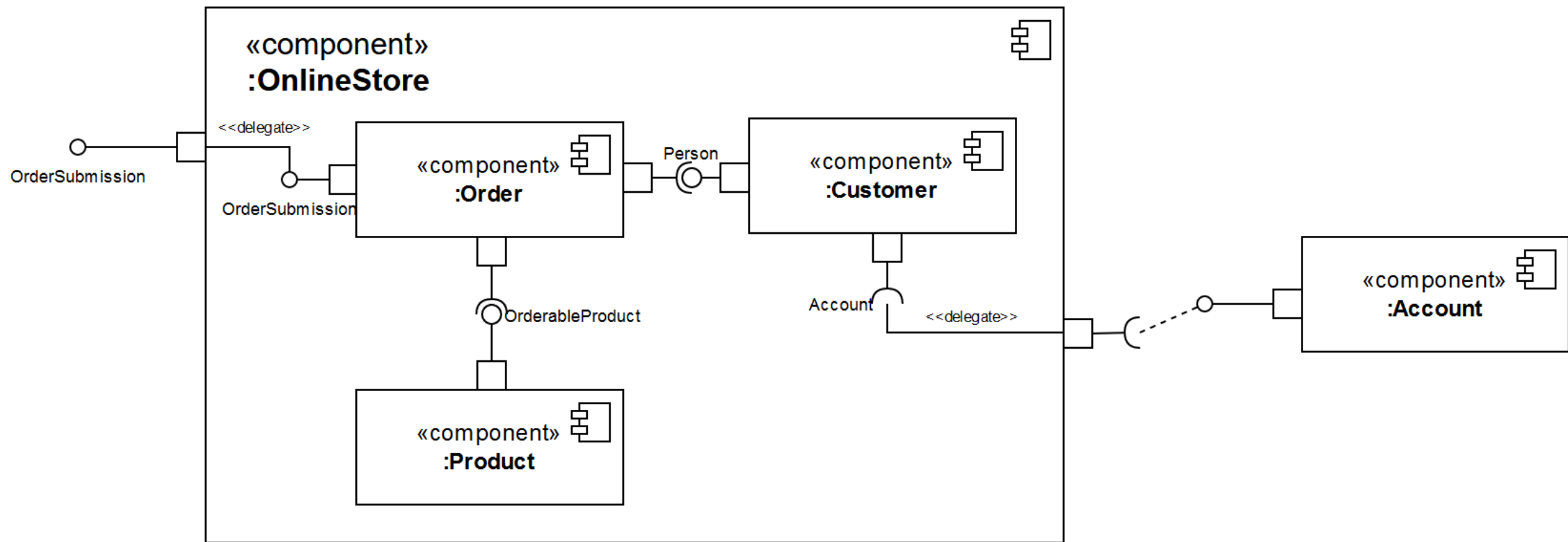
Components

- Components are the units of deployment. They are the smallest entities that can be deployed as part of a system.
- In Java, they are jar files.
- In Ruby, they are gem files.
- In .NET, they are DLLs.
- In compiled languages, they are aggregations of binary files.
- In interpreted languages, they are aggregations of source files.
- In all languages, they are the granule of deployment.

Component diagram parts



- Component symbol (Uml 1.0/2.0)
- Dependency
- Required and provided interfaces



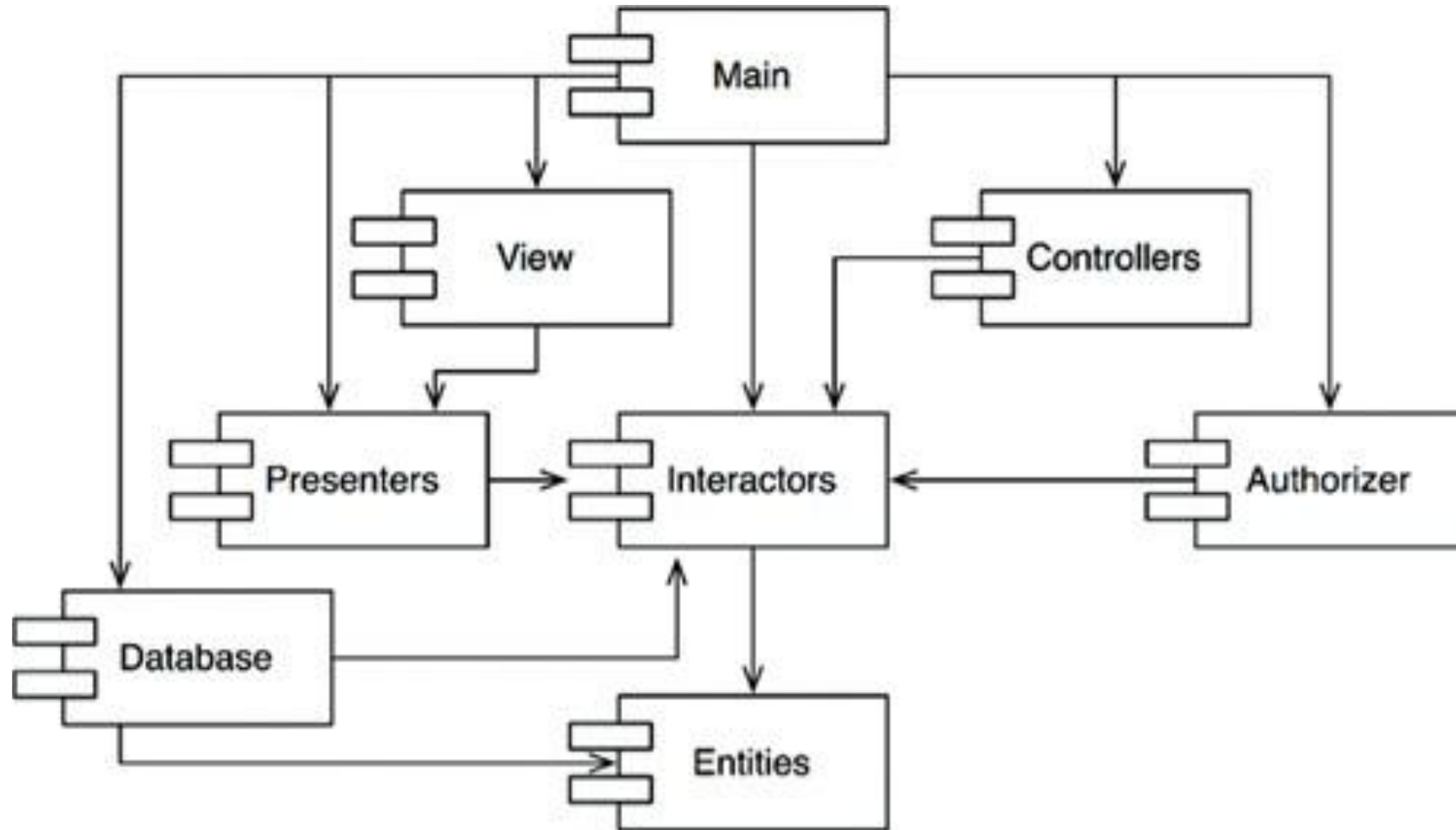
The Common Closure Principle (Generic SRP)

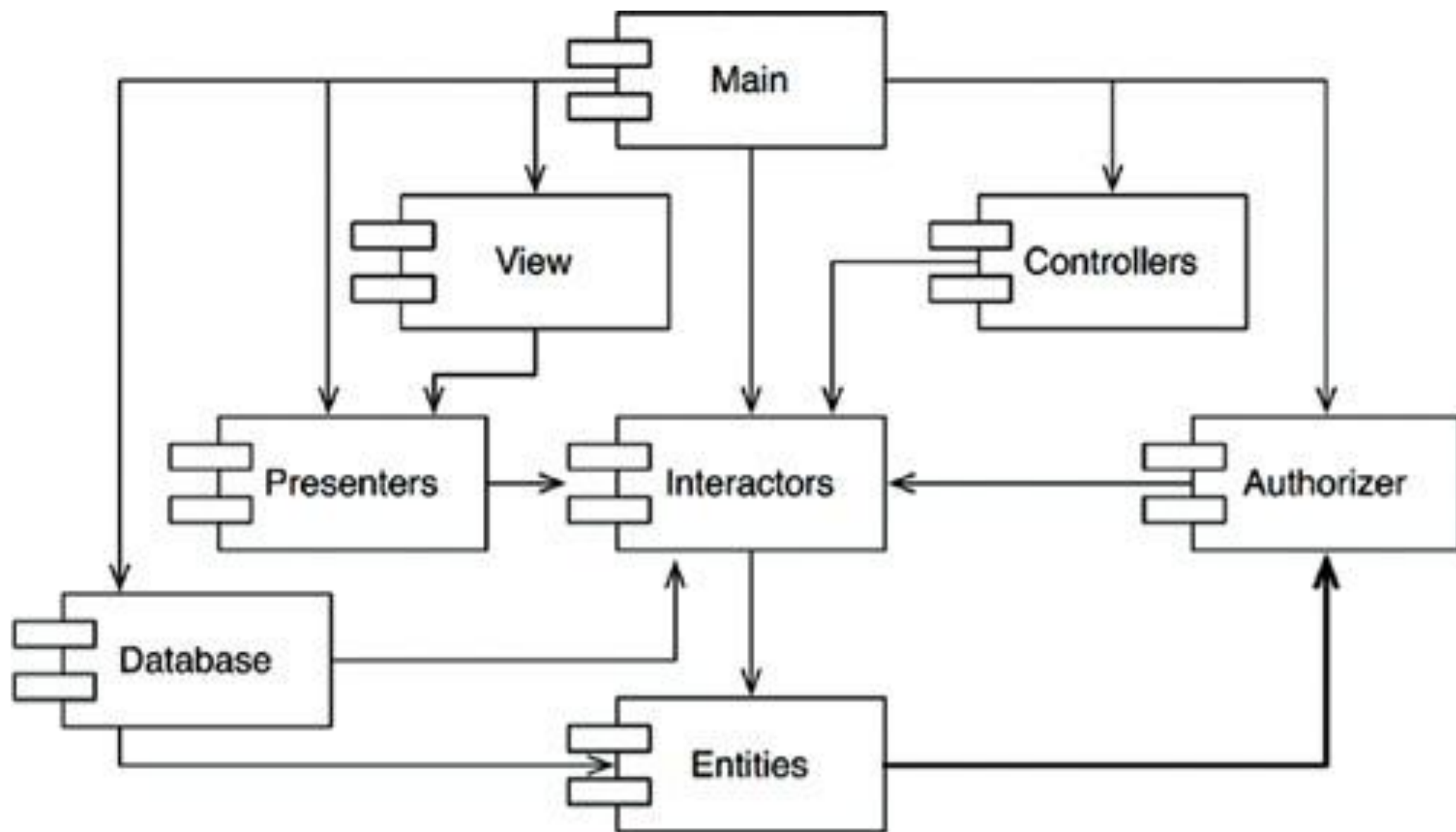
- Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change at different times and for different reasons.
- If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

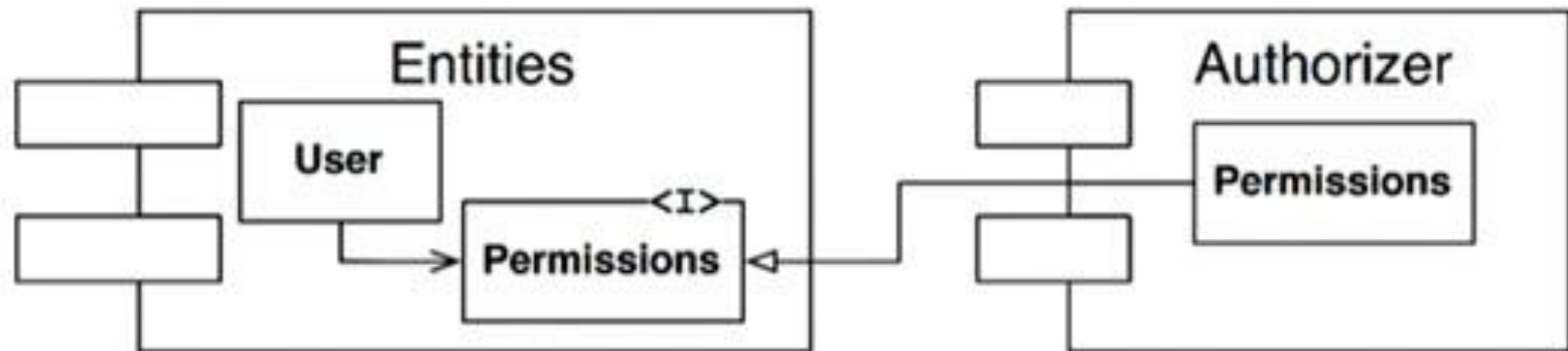
The Common Reuse Principle (Generic ISP)

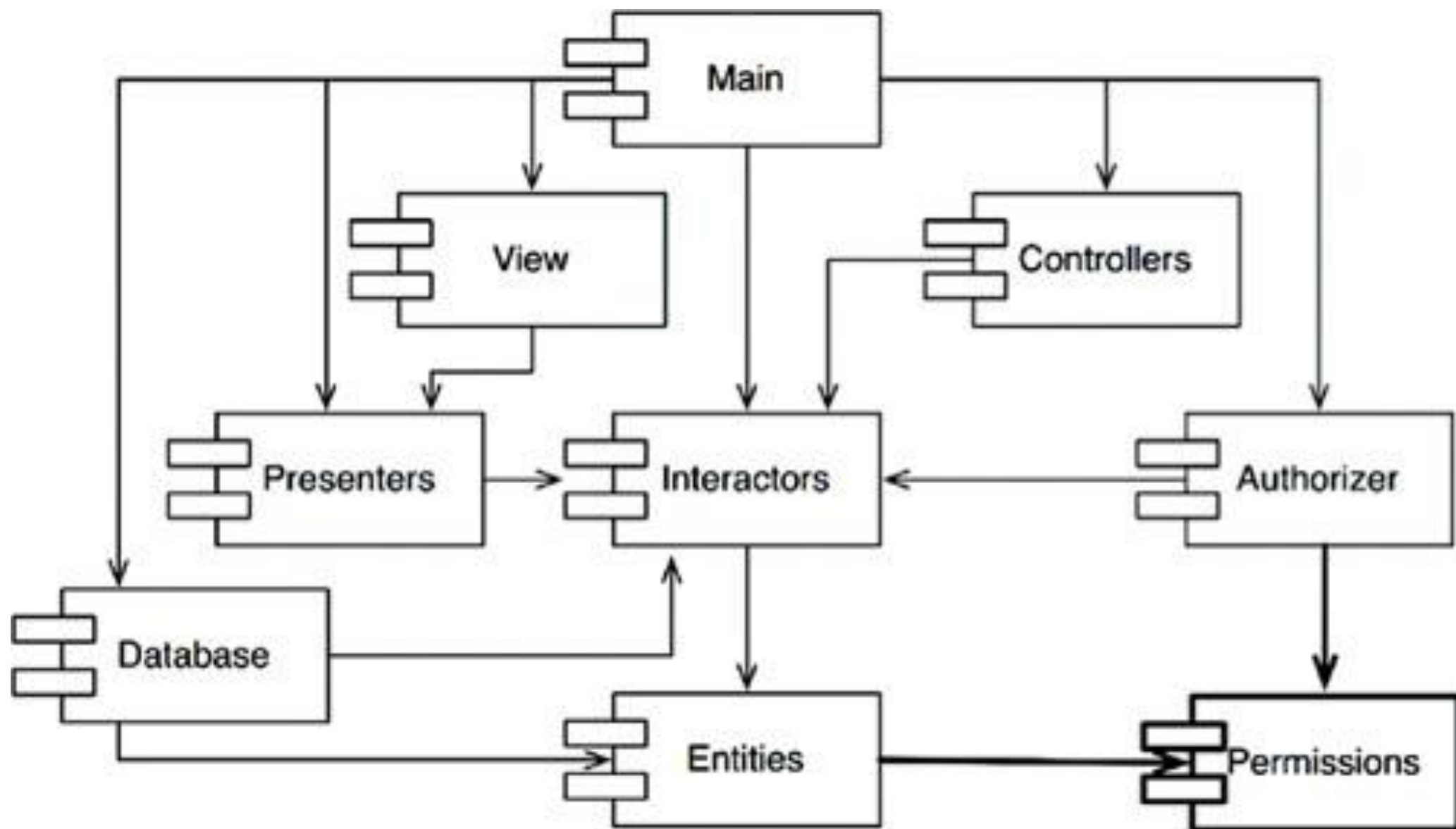
- “Don’t force users of a component to depend on things they don’t need.”
- When one component uses another, a dependency is created between the components. Perhaps the using component uses only one class within the used component, but that still doesn’t weaken the dependency. The using component still depends on the used component.
- Because of that dependency, every time the used component is changed, the using component will likely need corresponding changes. Even if no changes are necessary to the using component, it will likely still need to be recompiled, revalidated, and redeployed. This is true even if the using component doesn’t care about the change made in the used component.

Acyclic Dependencies Principle

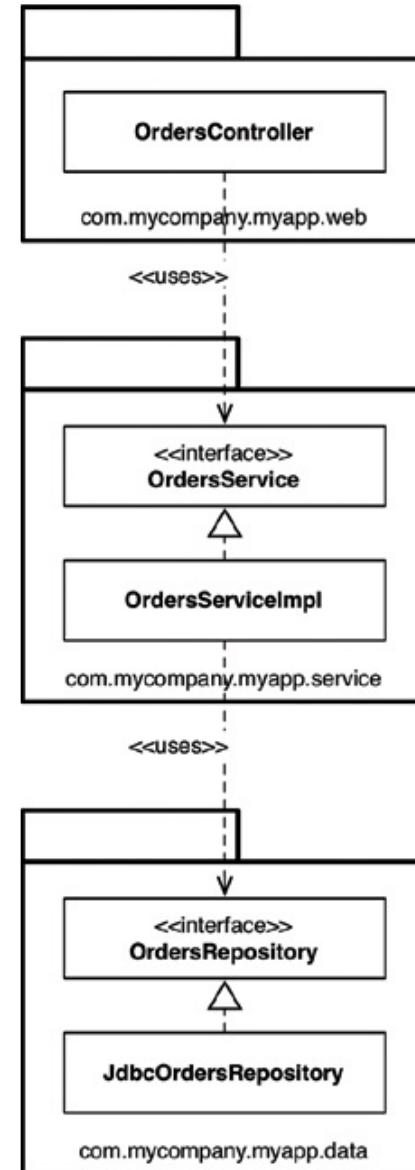




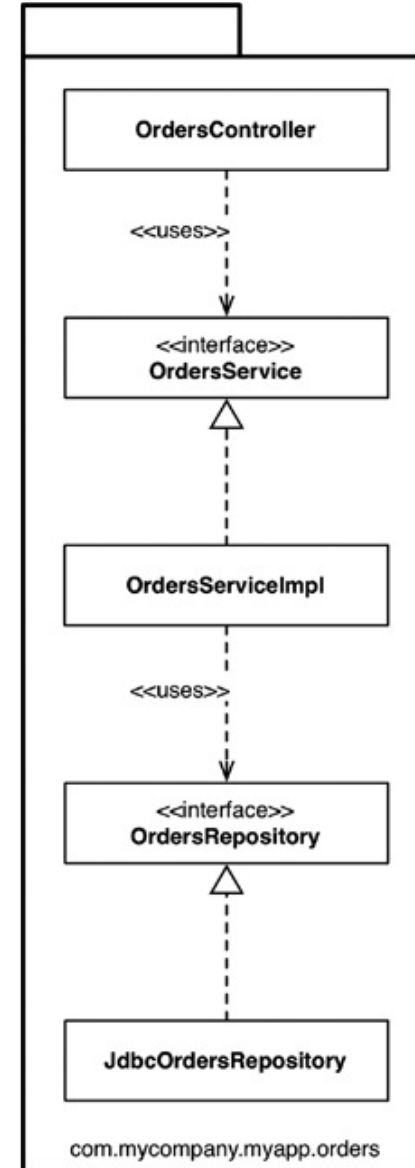




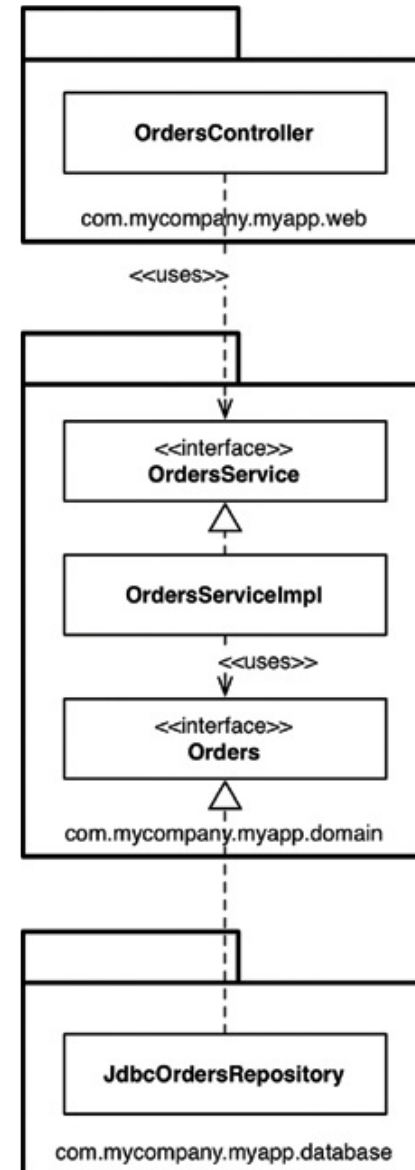
Package by layer (n tier)



Package by feature



Hexagonal architecture



Package by component

